DEIS Lab 1: ROS 2, Ubuntu, SSH, OpenCV
DT8007 VT26, Halmstad University

Binay Kumar Sah, binsah25@student.hh.se, Master's in Embedded and Intelligence System
Kabir Tamari, @student.hh.se, TAEIS 25

September, 2026

Introduction

This lab introduces the hardware and software stack that will be used throughout the project. The lab started with setting up a Raspberry Pi 4 and installing and running Ubuntu 24.04.3 LTS with ROS 2 Jazzy Jalisco installed on top. Lab 1 covers 5 different tasks, each covering the fundamentals of communication and ROS 2. Section 6 is optional work beyond the five tasks: running ROS 2 in Docker and streaming an ESP32-CAM into it.

1 Task 1: Trying Out the Raspberry Pi (Talker/Listener)

1.1 Objective

To confirm the connection between the Raspberry Pi and the local setup (PC) and to test ROS 2, Jazzy installation is operational.

1.2 Key Concepts

- **ROS 2:** ROS 2 is not an operating system, but a middleware that establishes a communication framework on top of DDS (Data Distribution Service). It is use to pass the messages between independent processes also called as **Nodes**.
- **Publisher and Subscriber:** the `talker` node periodically *publishes* string messages on a *topic*; the `listener` node *subscribes* to that same topic and prints every message it receives.
- **Environment sourcing,(Uderlay vs Overlay):** Before any `ros2` command works, the shell must be told where ROS 2 is installed via `source /opt/ros/jazzy/setup.bash`. This is a base layer also known as **Uderlay** whereas, **Overlay** is our own workspace where the packages we wrote are present and wraps up on top of base layer via `/ros2/install/setup.bash`
- **Demo packages:** `demo_nodes_cpp` and `demo_nodes_py` are pre-built example packages shipped with ROS 2, useful for a quick sanity check that does not require writing any code.

1.3 Observations and Results

We executed the given command, and Figure 1 shows the talker and listener communicating. Commands used:

```
# Terminal 1
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_cpp talker
```

```
# Terminal 2
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_py listener
```

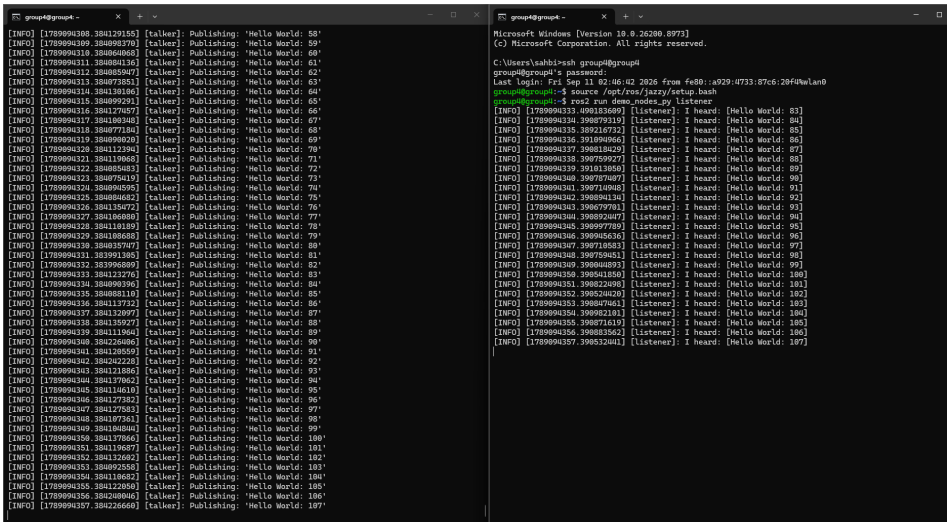


Figure 1: Talker and listener terminals confirming ROS 2 communication.

2 Task 2: SSH – Connecting Remotely to the Raspberry Pi

2.1 Objective

To install and enable an SSH server on the Raspberry Pi and connect to it remotely from a second computer, which is the standard way of interacting with a headless robot once it is no longer connected to a monitor and keyboard.

2.2 Observations and Results

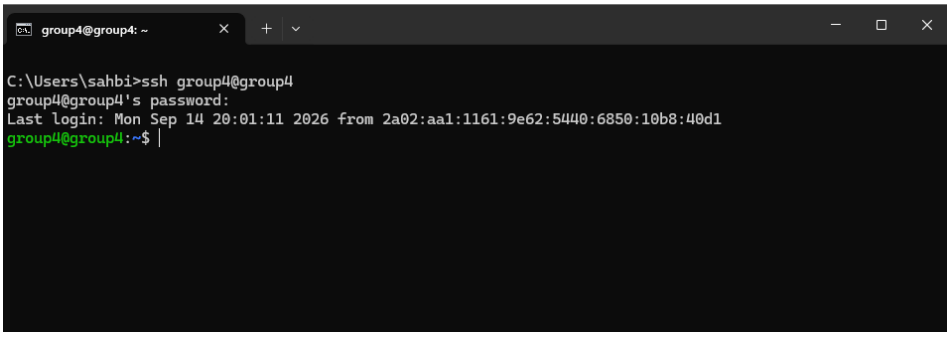


Figure 2: Successful SSH connection to the Raspberry Pi.

We follow the standard command to enable SSH in the Ubuntu system, and the figure 2 shows our successful SSH connection between Windows and Pi. Commands used on the Raspberry Pi:

```
sudo apt update && sudo apt upgrade
sudo apt install openssh-server
```

```
sudo systemctl enable --now ssh
sudo systemctl status ssh
```

Connecting from the remote computer:

```
ssh username@<raspberry_pi_ip_address>
```

3 Task 3: Image Processing on the Camera Feed

3.1 Objective

To familiarize with OpenCV and verify the working of the USB camera.

3.2 Key Concepts

- **OpenCV:** the de-facto standard open-source computer-vision library, used here outside of ROS 2 (in a Python virtual environment) to keep the first test simple.
- **Grayscale conversion:** `cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)` reduces a 3-channel colour image to a single intensity channel, which most classical edge detectors expect.
- **Canny edge detection:** `cv2.Canny(img, low, high)` finds edges by looking for strong intensity gradients between two threshold values; it is a fast, lightweight way to extract structure from a scene without deep learning.

3.3 Observations and Results

Given the camera test script provided only edge detection part , while we modify the code bit to generate a live feed in true color along with edge detection, as can be seen in figure 3a and 3b respectively. Test script used (`testWebCam.py`):

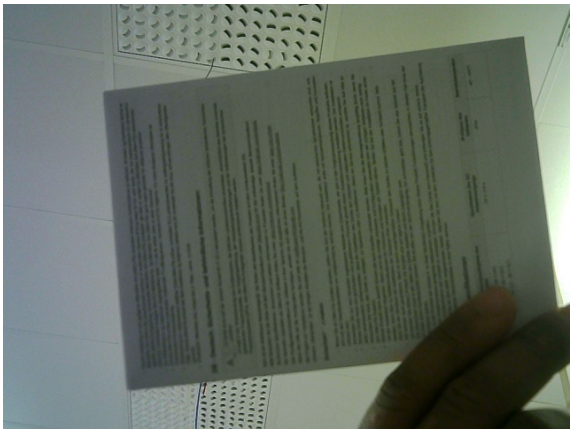
```
import cv2
cap = cv2.VideoCapture(0)
cv2.namedWindow('Color Feed')
cv2.namedWindow('Edge Detection')
while(True):
    ret, frame = cap.read()
    if(ret):
        img2 = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        edges = cv2.Canny(img2, 100, 200)

        cv2.imshow('', edges)
        cv2.imshow('',frame)

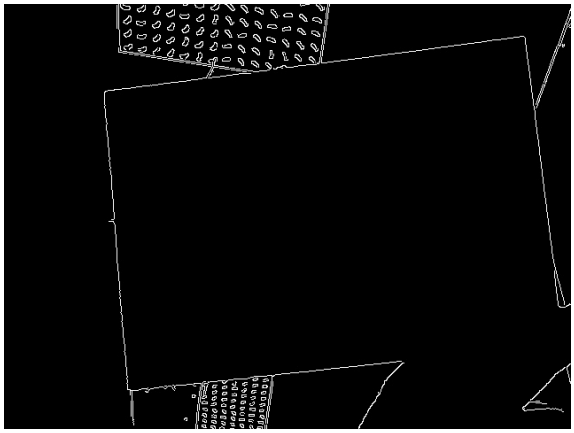
        key = cv2.waitKey(10) & 0xFF
        if key == ord('q'):
            print("Done")
            break
        elif key == ord('s'):
            cv2.imwrite("screenshot_color.jpg", frame)
            cv2.imwrite("screenshot_edges.jpg", edges)
        print("Took a Picture")
    else:
        break
cap.release()
cv2.destroyAllWindows()
```

Run with:

```
cd Desktop
source opencv-venv/bin/activate
python3 testWebCam.py
```



(a) Camera USB Live



(b) Canny edge detection applied to the live USB camera feed.

Figure 3: Displaying original camera feed and edge detection

4 Task 4: Building a ROS 2 Workspace and Package

4.1 Objective

To create a first custom ROS 2 workspace and package containing a minimal publisher and subscriber node, following the official ROS 2 beginner tutorial, in order to understand the standard project structure.

4.2 Key Concepts

- **Workspace (`colcon` workspace):** A directory (here we made `Labs/'Lab 1'/src`) that holds one or more ROS 2 packages and is built as a unit with the `colcon build` tool.
- **Package:** It is the basic unit of ROS 2 software organization; created here as an `ament_python` package, containing its own `package.xml`.
- **Publisher node:** a node that periodically publishes messages onto a named **topic** using a timer callback.
- **Subscriber node:** a node that registers a callback function which is invoked every time a new message arrives on the topic it is subscribed to.

4.3 Observations and Results

We follow the online tutorial for simple publisher and subscriber in cpp to create the publisher and subscriber note and run them in different terminals. The result is shown in figure 4

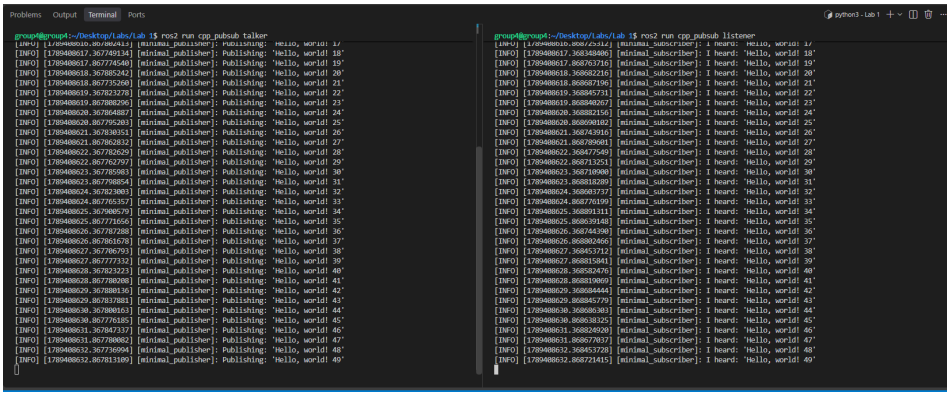


Figure 4: Custom ROS 2 publisher/subscriber package running successfully.

5 Task 5: Sending Motion Control Commands

5.1 Objective

To modify the listener to subscribe to the shared `/action` topic and publish the course's shared motor-command strings (forward, backward, left, right, half-speed forward, stop) directly from the command line.

5.2 Key Concepts

- **Message format:** a comma-separated payload of the form `timestamp,g,-1,-1,-2,leftSpeed;rightSpeed`, where the last two fields set the left and right wheel motor speeds.
- **Speed encoding:** `255` = full speed forward, `127` ≈ half speed, `0` = stopped, `-255` = full speed in reverse; independent left/right values enable in-place turning (e.g. `0;255` turns left).
- **One-shot publishing:** `ros2 topic pub -1 ...` publishes a single message and exits, which is convenient for sending discrete commands rather than a continuous stream.

5.3 Observations and Results

Commands sent (with listener running on `/action`):

```
# Forward
ros2 topic pub -1 /action std_msgs/String "data: <timestamp>,g,-1,-1,-2,255;255"
# Backward
ros2 topic pub -1 /action std_msgs/String "data: <timestamp>,g,-1,-1,-2,-255;-255"
# Left
ros2 topic pub -1 /action std_msgs/String "data: <timestamp>,g,-1,-1,-2,0;255"
# Right
ros2 topic pub -1 /action std_msgs/String "data: <timestamp>,g,-1,-1,-2,255;0"
# Forward (half speed)
ros2 topic pub -1 /action std_msgs/String "data: <timestamp>,g,-1,-1,-2,127;127"
# Stop
ros2 topic pub -1 /action std_msgs/String "data: <timestamp>,g,-1,-1,-2,0;0"
```

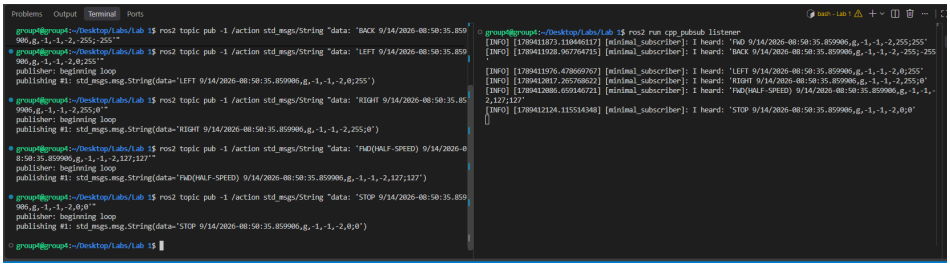


Figure 5: Listener receiving motor-control commands on the `/action` topic.

6 Optional: ROS2 Through Docker and ESP32-CAM Streaming into ROS 2

6.1 Objective

To run ROS 2 Jazzy in Docker on a laptop and stream an ESP32-CAM into it, since our research project is built around the ESP32.

6.2 Key Concepts

- **ROS 2 in Docker:** the image is based on `osrf/ros:jazzy-desktop`, the same Jazzy as on the Pi.
- **Host networking:** DDS discovery uses UDP multicast, which does not cross Docker's default bridge network, so the container uses `network_mode: host`.
- `ROS_DOMAIN_ID`**:** every group is on the same network, so we use domain 40 to keep our topics apart from theirs.
- **Shared memory:** Fast DDS sends same-host data through `/dev/shm`, which Docker keeps private to the container. `ipc: host` shares it with the host.
- **MJPEG stream:** the ESP32-CAM sends a continuous stream of JPEG frames over HTTP at `:81/stream`.

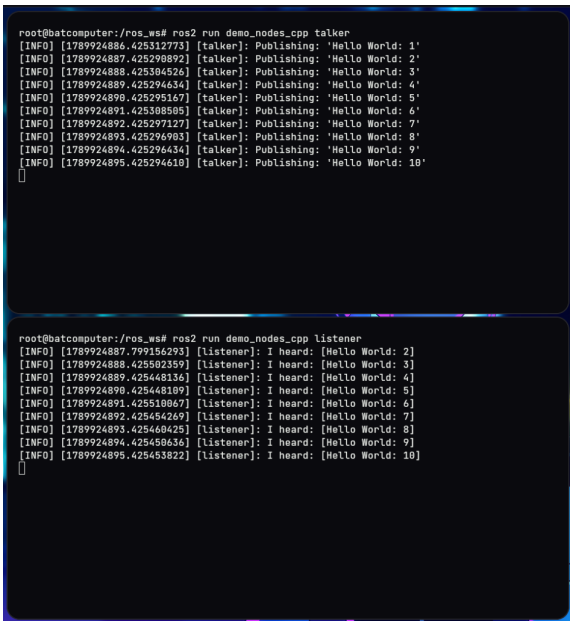
6.3 Observations and Results

We first repeated Tasks 1 and 4 inside the container to check the setup (Figure 6). The container is started with `docker compose up -d --build` from this compose file (trimmed):

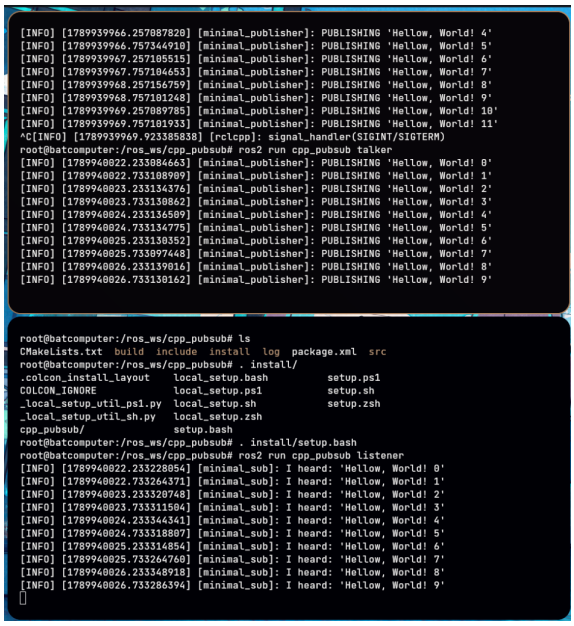
```
services:
  base:
    build: .
    network_mode: host
    ipc: host
    user: "ubuntu"
    environment:
      - DISPLAY=${DISPLAY}
      - QT_X11_NO_MITSHM=1
      - ROS_DOMAIN_ID=40
    volumes:
      - /tmp/.X11-unix:/tmp/.X11-unix:rw
      - ./ros_ws:/ros_ws
```

We wrote the ESP32-CAM firmware with ESP-IDF and flashed it with PlatformIO. Figure 7 shows its debug page with the live stream.

On the ROS side, our `esp_stream` package has two nodes (Figure 8). `cam_pub` reads the stream with OpenCV and publishes it on `/esp32/image/compressed`. `edge_detector` runs the same Canny



(a) Task 1: talker and listener.



(b) Task 4: our `cpp_pubsub` package.

Figure 6: Tasks 1 and 4 repeated inside the Docker container.

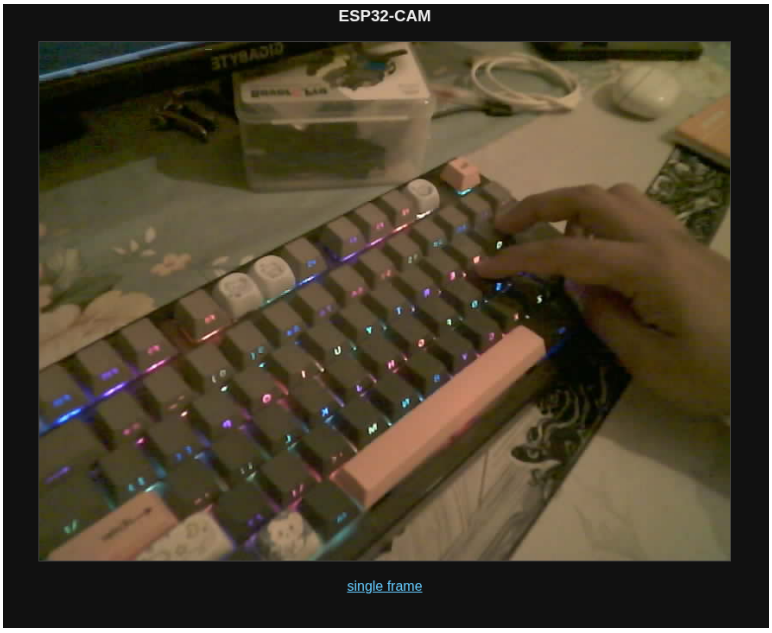


Figure 7: ESP32-CAM debug page showing the live stream.

edge detection as Task 3 and publishes the result on `/esp32/edges/compressed`.

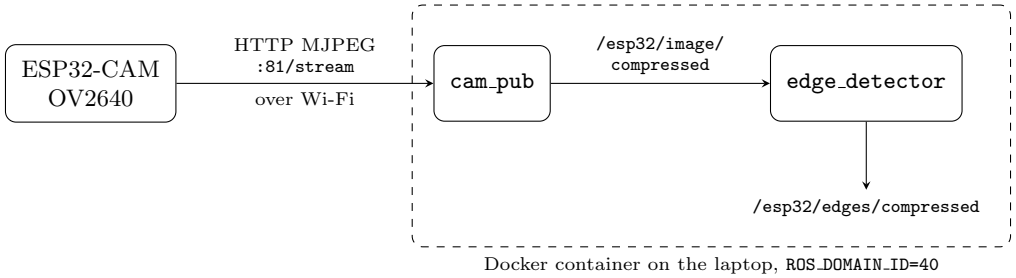


Figure 8: The ESP32-CAM pipeline.

Commands used inside the container:

```
cd /ros_ws/esp_stream
colcon build --packages-select esp_stream
source install/setup.bash

# Terminal 1
ros2 run esp_stream cam_publisher --ros-args -p stream_url:=http://192.168.0.70:81/stream -p
    fps:=12
# Terminal 2
ros2 run esp_stream edge_detector
# Terminals 3 and 4
ros2 topic hz /esp32/image/compressed
ros2 topic hz /esp32/edges/compressed
```

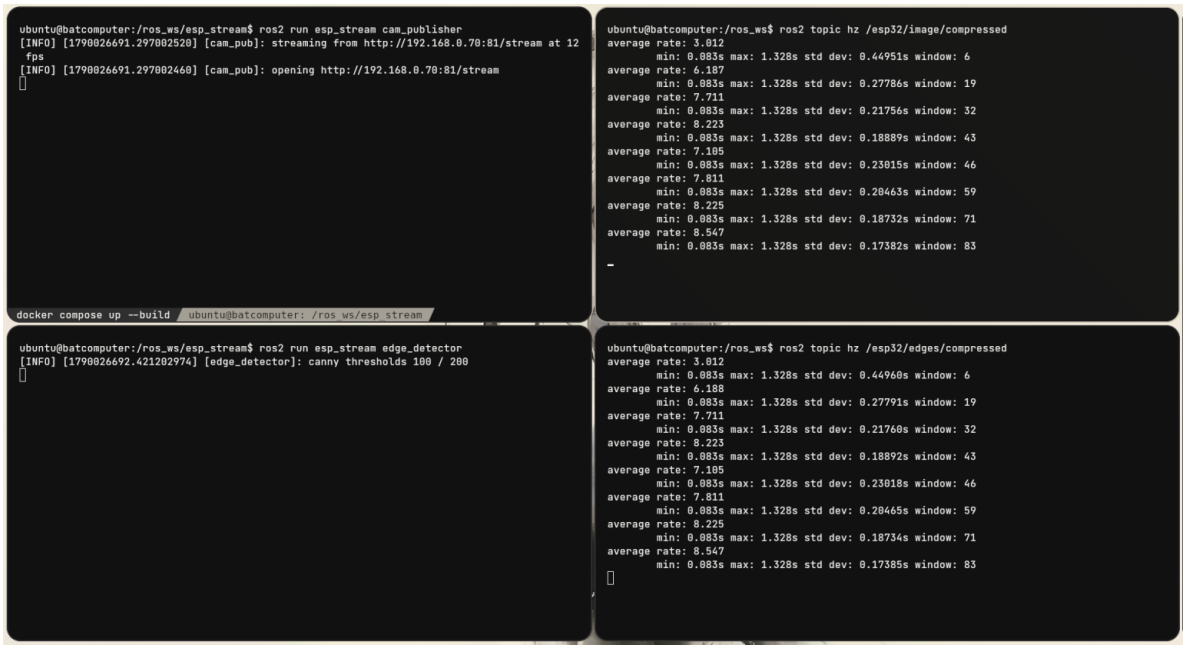


Figure 9: Both nodes running (left) and `ros2 topic hz` on the image and edge topics (right).

Both topics run at about 8.5 messages per second against the 12 fps target (Figure 9). The missing frames are lost due to ESP32 cam heating up and dropping it's processing power.

7 Conclusion

Thus from Lab 1 we learn the baseline of ROS2 setup and it's architecture layers used in robot software stack. We also learn to join the shared network and run the ROS 2 Jazzy correctly. Furthermore, openCV basic vision pipeline demonstrated that the camera and image-processing tools are functional outside of ROS 2. We end this lab by sending commands on the shared `/action` topic provided first-hand exposure to the motor-control protocol that will let the group's robot both move and, later, interoperate with other groups' robots.

In the optional part, we streamed an ESP32-CAM into ROS 2 running in Docker, which our research project will build on.

Contribution table

	Binay Kumar Sah	Kabir Tamari
Task 1 (Talker/Listener)	50%	50%
Task 2 (SSH)	100%	0%
Task 3 (OpenCV)	50%	50%
Task 4 (Workspace/Package)	50%	50%
Task 5 (Motion Commands)	100%	0%
Optional: Docker, networking, shared memory	0%	100%
Optional: ESP32-CAM firmware and ROS 2 bridge	0%	100%
Report writing	80%	20%

Table 1: Group member contribution breakdown